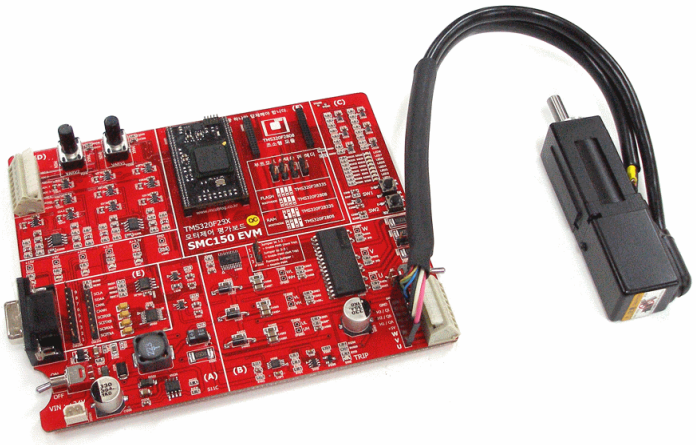
* 본 자료는 모터구동의 한 예제로만 사용하십시오. 어플리케이션 적용으로 인한 문제에 대해서는 싱크웍스에서 책임지지 않습니다. 또한, 기술적인 지원도 할 수 없음을 알려드립니다.

* 본 예제 프로그램은 Texas Instruments(TI)사에서 TMS320F28X 프로세서 사용자들을 대상으로 공개한 Digital Motor Control Library를 기초로 만들어졌으며, SMC150 개발보드에 적합하도록 일부 수정되었습니다. (Digital Motor Control Library 최신버전은 TI의 ControlSUITE를 통해 받아보실 수 있습니다.)

*Revision 2012.07.30 (v1.5)

Incremental Encoder를 이용한 3상 PMSM의 속도/위치 제어



*Revision History

날 짜	내 용
12. 01. 12.	- V1.0 / 초판
12. 07. 30.	- V1.5 / CCS V5용 예제 프로젝트 추가, TMS320F28069용 예제 추가

목차

1 SMC150 개발보드 환경설정 4

1.1 SMC150 개발보드에 PMSM 연결하기 5
1.2 PMSM 사양 및 정격/성능 6
1.3 전원 공급 예 7
1.4 PWM 출력 및 TRIP 제어용 점퍼 핀-헤더 설정 (JP3000) 8

2 예제 실습 10

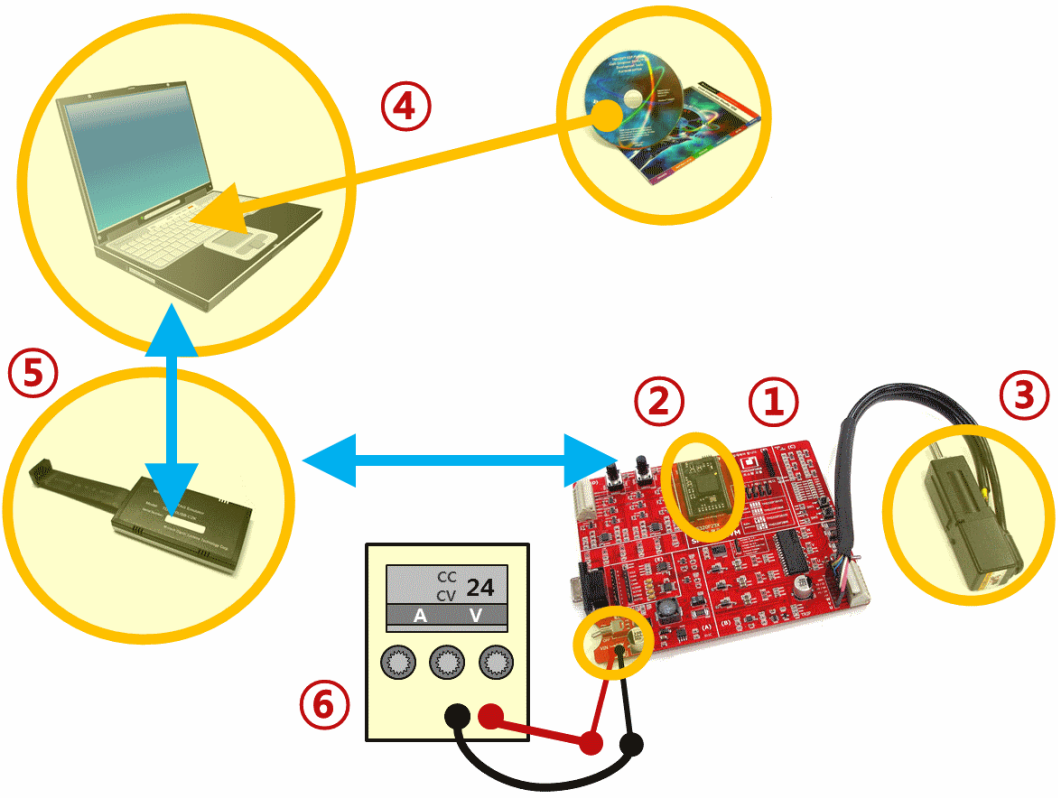
2.1 예제 디렉터리 색인 10
2.2 예제 구성 12
2.3 실습 따라 하기 (BUILD LEVEL 1) 13
2.4 실습 따라 하기 (BUILD LEVEL 2) 16
2.5 실습 따라 하기 (BUILD LEVEL 3) 19
2.6 실습 따라 하기 (BUILD LEVEL 4) 22
2.7 실습 따라 하기 (BUILD LEVEL 5) 24
2.8 실습 따라 하기 (BUILD LEVEL 6) 27

3 실습 끝내기 절차 29

1 SMC150 개발보드 환경설정

예제 프로그램을 실행하기 위해 아래의 개발환경들이 필요합니다.

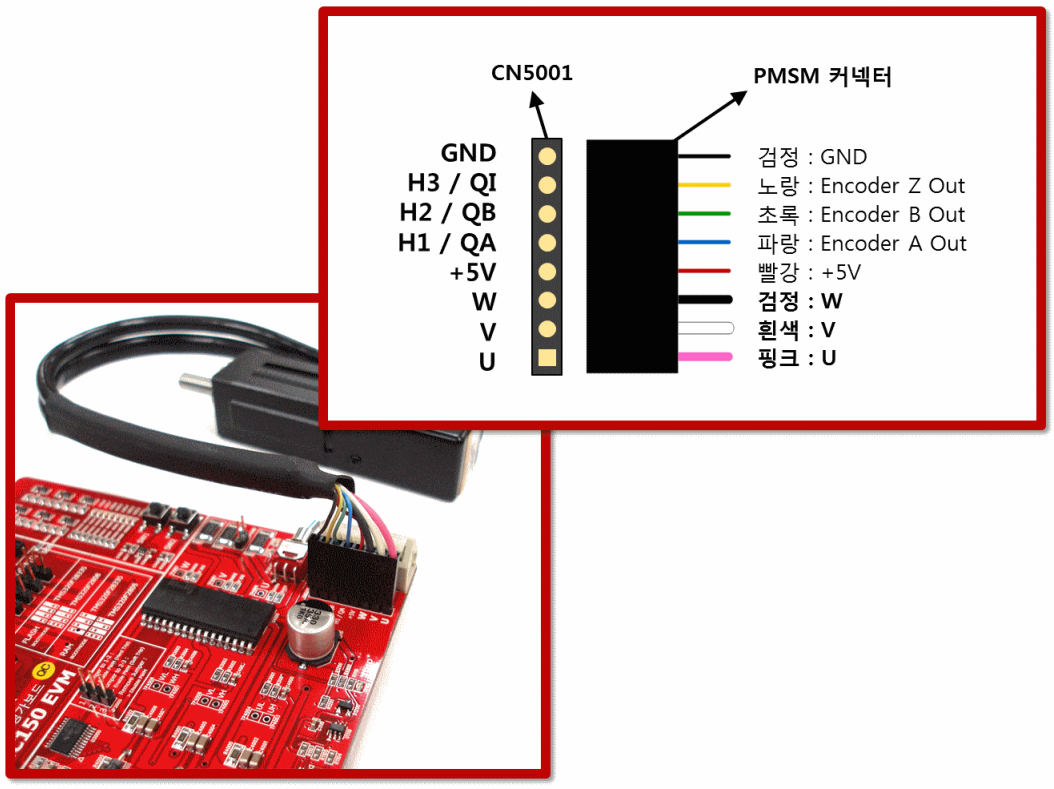
① **SMC150 개발보드** (3상 모터제어 개발보드)
② **프로세서 모듈** (TMS320F28335 초소형, TMS320F2808 초소형)
③ **26W 3Phase PMSM** (with 2,000C/T Incremental Encoder)
④ **Code Composer Studio**(v3.3 혹은 그 이상의 버전)가 설치된 IBM 호환 PC
⑤ **JTAG 에뮬레이터** (TDS510USB-C2K 혹은 동종의 JTAG 에뮬레이터)
⑥ **DC 전원공급기(DC Power Supply) 1개** (DC +24V 공급이 가능한 제품)



[그림 1-1] – 예제 프로그램 실행을 위한 개발환경

1.1 SMC150 개발보드에 PMSM 연결하기

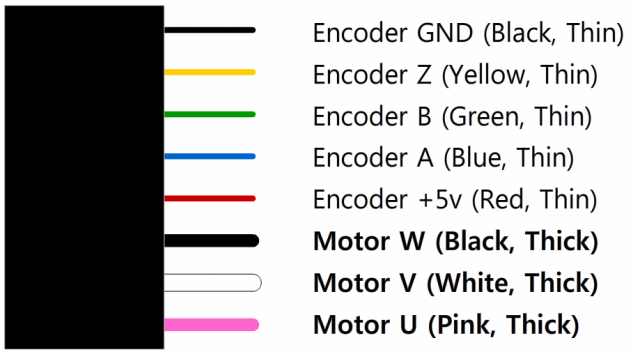
SMC150 개발보드와 함께 판매되는 Permanent Magnet Synchronous Motor(PMSM, BLAC)는 26W 급의 3상 제품이며, 2,000C/T의 고-분해능 엔코더를 내장하고 있습니다. 이 PMSM은 아래의 그림 1-2와 같이 SMC150 개발보드의 **CN5001 핀-헤더**에 연결할 수 있습니다.



[그림 1-2] – SMC150 개발보드와 PMSM의 연결

1.2 PMSM 사양 및 정격/성능

SMC150 개발보드와 함께 판매되는 3상 PMSM은 2,000C/T 분해능의 Incremental Encoder(A/B/Z)를 내장하고 있으며, 각 신호 및 전원과 Lead-wire 색의 관계는 아래의 그림 1-3과 같습니다.



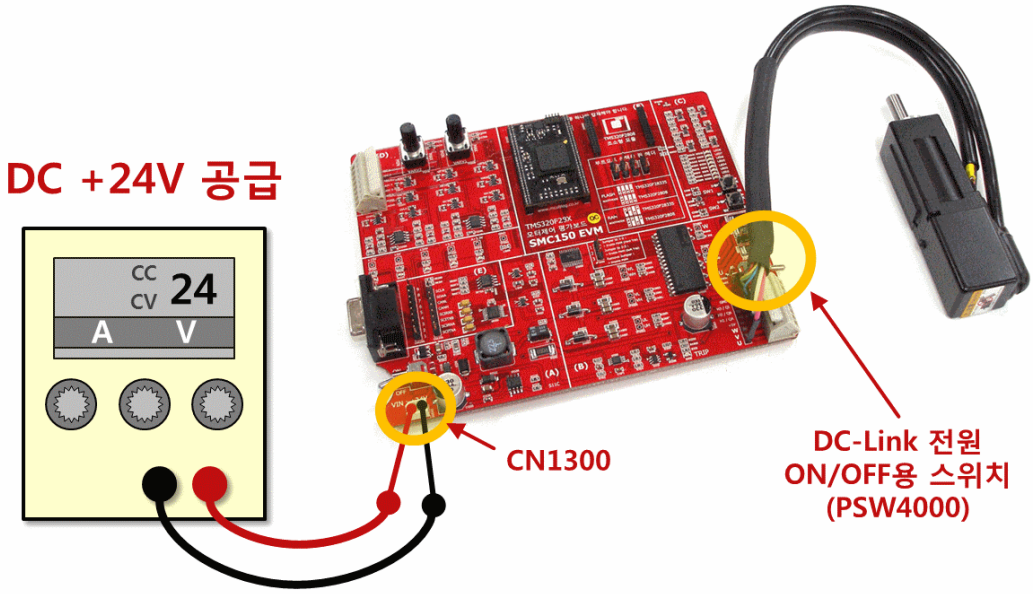
[그림 1-3] – PMSM 커넥터 정의

모터	사양
Motor Form.	P.M. 8Poles, 3Ph. Y-Connect
Voltage	DC 24V
Rated Current	2.9A
Rated Power	26W
Rated Speed	4,000RPM
Rated Torque	0.062N.m {0.63kgf•cm}
Max. Torque	0.185N.m {1.89kgf•cm}
Stall Torque	0.062N.m {0.63kgf•cm}
No Load Speed	5,000RPM
Rotor Moment of Inertia	0.004gf•cm•s^2
Mounting Flange	Φ 22
Approximate Mass	190g
No Operating Temp. Range	-10℃ ~ +85℃
Operating Temp. Range	0℃ ~ +40℃

엔코더	사양
Rated Voltage	DC 5V
Resolution	2,000 C/T
Outputs	A, B, Z(Index)

[표 1-1] – SMC150 개발보드와 함께 판매되는 PMSM의 사양 및 정격

1.3 전원 공급 예



[그림 1-4] – SMC150 개발보드의 전원 공급 예

주의: 전력 변환이나 모터 구동과 같은 어플리케이션은 고-전압 혹은 고-전류에 노출되어 있기 쉽습니다. 이는 부주의로 인한 사고 발생시 장비들을 손상시킬 뿐만 아니라 여러분에게도 위험을 줄 수 있습니다. 따라서 실습을 진행함에 있어 명시된 절차를 필히 지켜주셔야 합니다. 또한 전류 수치의 확인 및 제한이 용이한 DC 전원공급기의 사용을 권고 드리며, 안전한 실습을 위해, 예제 프로그램의 실행을 멈추기 전, 반드시 DC-Link에 공급된 전원을 차단해주십시오. (PSW4000 스위치 사용)

1.4 PWM 출력 및 Trip 제어용 점퍼 핀-헤더 설정 (JP3000)

SMC150 개발보드에 탑재된 TMS320F28X 프로세서의 3.3V Level PWM 출력 신호들은 Voltage Translator(SN74LVC4245A, U3000)를 거쳐, 5.0V Level로 가공되고, 모터 드라이버 IC(Smart Power Module, U4000)로 전달됩니다. 여기에 사용된 Voltage Translator는 /OE 핀을 가지고 있어, 해당 핀의 제어에 따라, 모터 드라이버 IC로 PWM을 전달(Enable)하거나 혹은 차단(Disable)할 수 있습니다. 또한, SMC150 개발보드에는 Texas Instruments사의 INA200(High-Side Measurement Current-Shunt Monitor with Open-Drain Comparator and Reference) 소자를 이용한 과-전류 Trip 회로가 설계되어 있습니다. 이 회로는 DC-Link에 흐르는 전류가 설정된 수치(1A)를 초과할 경우, Trip 펄스를 발생시키게 됩니다. 이렇게 발생된 Trip 펄스는 TMS320F28X 프로세서에 전달되며, 동시에 JP3000 점퍼 핀-헤더(2.54mm 1x3)를 거쳐서, PWM 출력을 모터 드라이버 IC로 전달하는 Voltage Translator의 /OE 핀에도 전달되게 됩니다.

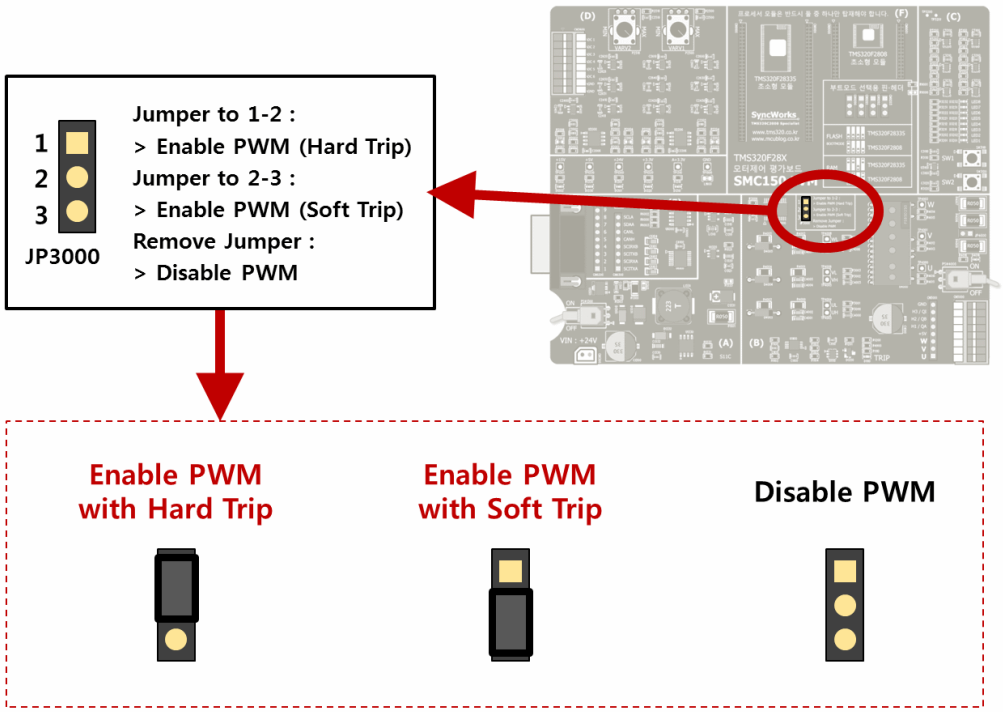
즉, 모터 제어 중 DC-Link에 흐르는 전류가 설정된 수치를 초과하면, 과-전류 Trip 회로에 의해 발생된 Trip 펄스가 TMS320F28X 프로세서의 PWM 출력을 모터 드라이버 IC에 전달하는 Voltage Translator를 Disable 하도록 설정할 수 있습니다. (Hard-Trip)

만약, 이 Trip 펄스가 Voltage Translator(SN74LVC4245A)의 /OE 핀에 직접 전달되지 않도록 설정하면, TMS320F28X 프로세서가 Trip 펄스를 감지하고, PWM 핀의 상태를 가장 안정한 상태로 고정하거나 범용 입출력 포트(GPIO) 조작을 통해 Voltage Translator를 Disable 하도록 할 수 있습니다. (Soft Trip)

이런 모든 설정은 JP3000 점퍼 핀-헤더(2.54mm 1x3)의 조작에 따라 이루어지며, 제품과 함께 제공되는 2P 점퍼로, PWM 출력을 Disable / Enable with Hard Trip / Enable with Soft Trip의 총 3가지로 제어할 수 있습니다.

예제 실행을 위해, 그림 1-5와 표 1-2를 참조하여, PWM 출력이 모터 드라이버 IC(Smart Power Module)에 전달될 수 있도록 해주십시오. Hard Trip 동작과 함께 PWM 출력을 Enable 할 경우, SMC150 개발보드에 설계된 과-전류 Trip 회로 출력에 의해, 자동으로 전류제한 동작이 이루어지며, Soft Trip을 선택하면, 예제에 의해 소프트웨어적으로 전류제한 동작이 이루어집니다.

JP3000 핀-헤더를 통해, PWM 출력이 Enable 되지 않은 경우, 예제를 실행해도 모터가 돌아가지 않습니다.



[그림 1-5] – JP3000 점퍼 핀-헤더의 위치 및 설정 예

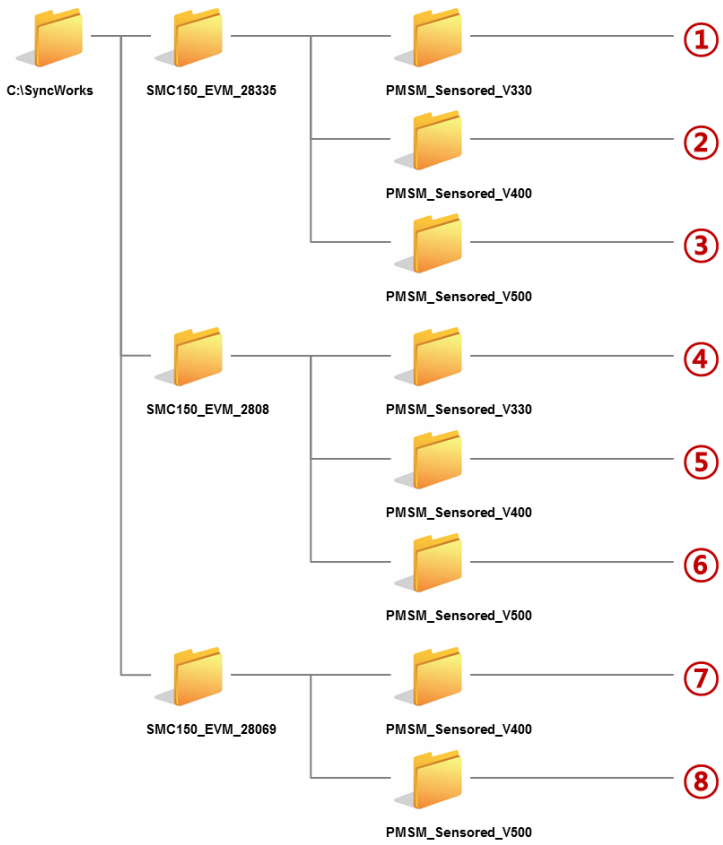
JP3000 핀-헤더	TMS320F28335	TMS320F2808	TMS320F28069 (어댑터를 통해 장착)
1 번 핀	과-전류 Trip 회로의 출력 펄스		
2 번 핀	Voltage Translator(SN74LVC4245A, U3000)의 /OE 핀		
3 번 핀	GPIO14/-TZ3/ -XHOLD/SCITXDB/MCLKXB	GPIO6/EPWM4A/ EPWMSYNCI/EPWMSYNCO	GPIO42/EPWM8A/ -TZ1/COMP1OUT

[표 1-2] – JP3000 점퍼 핀-헤더의 정의

2 예제 실습

SMC150 개발보드의 활용예제는, TMS320F28X 프로세서를 이용하여, 3상 BLDCM이나 PMSM의 토크, 속도, 위치 등을 제어해볼 수 있도록 구성되어 있으며, Texas Instruments(TI)사의 Digital Motor Control Library를 기초로, SMC150 개발보드에 적합하도록 일부 수정한 것 입니다. BLDCM의 경우 Hall-Effect Sensor가 주는 정보를 토대로 모터의 속력을 알아내고, 여기에 PI제어기를 붙여서, 사용자가 설정한 속력을 추종하는 예제가 제공되며, PMSM은 2,000C/T의 고-분해능 엔코더와 벡터 제어 기법(FOC)을 통해, 모터의 토크, 속도, 위치 등을 제어(PI)해볼 수 있도록 하는 예제가 제공되고 있습니다. 추후에도 사용자들의 요구에 맞춰, 다양한 활용예제들이 추가될 예정입니다. - TI의 최신 예제 및 라이브러리는 TI 웹 페이지가 아닌 ControlSUITE(클릭) 통해서 받으실 수 있습니다.

2.1 예제 디렉터리 색인



[그림 2-1] – PMSM Sensored Control 예제 디렉터리 색인

그림 2-1의 1번과 2번, 3번 경로는 TMS320F28335 초소형 모듈 사용자를 위한 예제 프로젝트가 담겨있으며, PMSM_Sensored_V330 경로에는 CCS V3.3용, PMSM_Sensored_V400 경로에는 CCS V4.2.X용, PMSM_Sensored_V500 경로에는 CCS V5.2.X용 예제가 준비되어 있습니다.

4번과 5번, 6번 경로에는 TMS320F2808 초소형 모듈 사용자를 위한 예제 프로젝트가 담겨 있으며, 앞의 경우와 마찬가지로 XXX_V330은 CCS V3.3용 예제 프로젝트이고, XXX_V400은 CCS V4.2.X용 예제 프로젝트, XXX_V500은 CCS V5.2.X용 예제 프로젝트 입니다.

7번과 8번 경로에는 전용 어댑터를 통해 SMC150 개발보드에 장착되는 TMS320F28069 보급형 모듈 사용자용 예제 프로젝트가 담겨 있습니다. CCS 버전에 따른 경로 구분은 앞서의 경우와 동일합니다. 단, CCS V3.3의 경우, TMS320F28069 프로세서를 정식으로 지원하지 않기에 CCS V3.3용 예제는 제외되었습니다. ("C:\SyncWorks\TIDCS\..." 이하의 폴더에는 예제 실행에 필요한 TI 제공 칩-지원 헤더 및 소스가 위치하며, 그림 2-1에서는 생략되어 있습니다. – 예제 실행을 위해 반드시 설치해야 합니다.)

2.2 예제 구성

SMC150 개발보드용 PMSM Sensored Control 예제는 총 6단계의 Build Level로 구분되어 있으며, 각 단계는 아래와 같습니다. Build Level 설정에 따라, 그에 적합한 코드가 Compile 되도록 구성되어 있으며, Build Level 설정은 예제 프로그램의 헤더(PMSM_Sensored.h)파일에 정의되어 있습니다.

- **LEVEL 1 (모터를 구동하지 않는 단계)**
  이 단계는 SMC150 개발보드와 모터를 연결하지 않고, TMS320F28X 프로세서에서 Space-Vector PWM 생성용 S/W 모듈을 실행하여, PWM 출력이 정상인지 확인하는 단계 입니다.

- **LEVEL 2**
  PMSM을 Open-loop로 구동하면서, ADC를 통해 U상과 V상의 전류가 올바르게 검출되는지 확인하는 단계 입니다.

- **LEVEL 3**
  PMSM을 Open-loop로 구동하면서, ADC를 통해 검출된 U, V상 전류정보에 PI 제어 모듈을 연결하고, 전류제어가 올바르게 되는지 확인하는 단계 입니다.

- **LEVEL 4**
  PMSM을 Open-loop로 구동하면서, TMS320F28X 프로세서의 EQEP 모듈로 엔코더로부터의 신호를 해석하고, 올바르게 위치정보를 검출해내는지 확인하는 단계 입니다.

- **LEVEL 5**
  1~4까지의 Build Level에서 확인한, 전류제어 Loop, 엔코더의 위치정보 등을 기초로 하여, PMSM을 Closed-loop로 제어하는 단계이며, Q축 전류제어 Loop 외부에 속도제어 PI 제어 Loop를 추가하여, 사용자의 속도지령을 추종하도록 합니다.

- **LEVEL 6**
  Build Level 5와 유사한 Closed-loop 단계이며, Q축 전류제어 Loop 외부에 속도 PI 제어 Loop 대신, 위치제어 Loop가 구성되어 있어, 모터의 회전자가 사용자의 위치지령을 추종하도록 합니다.

2.3 실습 따라 하기 (Build Level 1)

① SMC150 개발보드에 DC+24V의 전원을 공급하고, JTAG 에뮬레이터를 통해, SMC150 개발보드에 장착된 프로세서 모듈과 PC의 Code Composer Studio를 연결합니다.

② SMC150 개발보드의 PSW1300 스위치를 "ON" 위치로 조작하여, 전원을 공급합니다. 단, DC-Link 전원 공급을 제어하는 PSW4000 스위치는 켜지 않습니다. (개발보드 조작과 관련한 세부내용은 제품 매뉴얼에서 자세히 확인하실 수 있습니다.)

③ Code Composer Studio에서 예제 프로젝트를 열고, PMSM_Sensored.h 파일에 정의된 Build Level이 "LEVEL1" 인지 확인합니다. (다르다면 LEVEL1로 수정합니다.)

④ 예제 프로젝트를 Build하고, 프로세서 모듈에 Output File을 Load 합니다.

⑤ Reset CPU → Restart → Enable Real-time Mode → Run(or Resume) 명령을 실행합니다.

⑥ 예제 프로젝트가 위치한 경로에 포함된 이미지 파일(*.png)을 참조하여, Watch Window에 관찰 및 조작할 변수를 등록하고, Graph Window를 설정합니다. CCS V3.3용 예제의 경우 GraphProperties1.png, GraphProperties2.png 파일을 참조하여 직접 Graph Window를 설정해야 하며, CCS V4.2.X / CCS V5.2.X용 예제는 Import 기능을 통해 미리 설정된 값들을 불러들일 수 있도록 GraphProperties1.graphPorp, GraphProperties2.graphPorp 파일이 포함되어 있습니다. Watch Window에 등록할 변수들은 CCS V3.3용 예제와 CCS V4.2.X용 예제 모두 이미지 파일로 제공되는 예시(WatchWindow_BuildLevel1.png)를 참조하여 직접 변수들을 등록해줘야 합니다.) 단, CCS V5.2.X의 경우는 미리 저장된 등록대상 변수들을 Watch Window(Expressions 탭)에 Import 할 수 있습니다. WatchWindow_BuildLevel1.txt 파일을 통해 관찰대상 변수들을 한번에 등록하십시오. – Expression 탭의 빈 공간을 마우스 우측버튼으로 클릭하면, 단축메뉴에서 Import 항목을 찾을 수 있습니다.

⑦ Watch Window와 Graph Window 모두 Continuous Refresh 옵션을 활성화 합니다.

⑧ DC-Link 전원 제어용 스위치(PSW4000)는 "OFF' 위치에 두고 조작하지 않습니다.

⑨ Watch Window 에 등록된 "EnableFlag" 변수를 0 에서 1 로 변경합니다. (Build Level 1 에서는 모터를 SMC150 개발보드와 연결하지 않습니다.)

⑩ Build Level 1은 SMC150 개발보드와 모터를 연결하지 않고, TMS320F28X 프로세서에서 Space-Vector PWM 생성용 S/W 모듈(SVGEN_DQ)을 실행하여, PWM 출력이 정상인지 확인하는 단계 입니다.

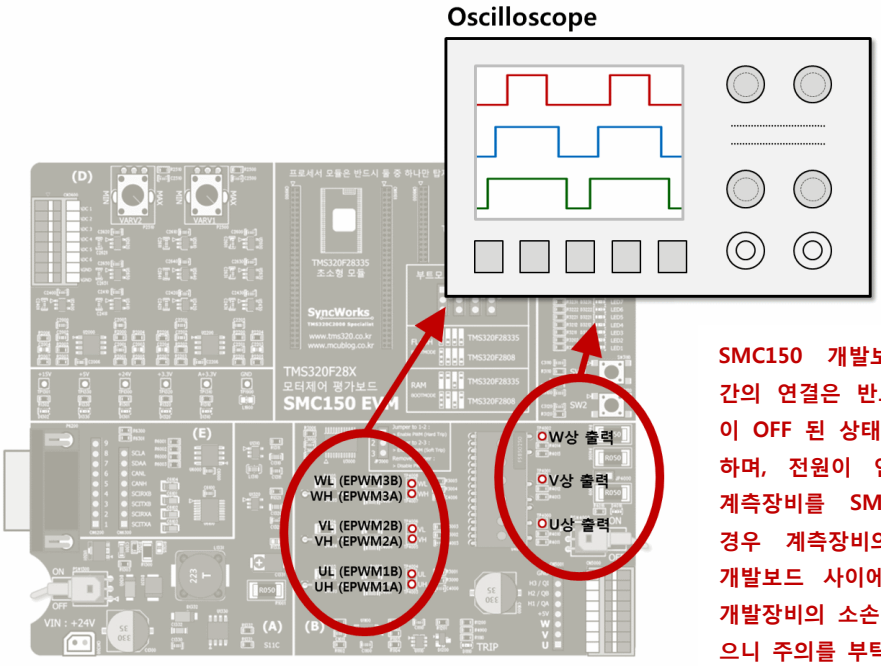
- **조작 대상 변수 → SpeedRef, VqTesting**

  이 단계에서는 실제 모터는 구동하지 않고, Open-loop Algorithm을 실행하여, PWM 출력을 관찰하는 단계 입니다. SpeedRef 변수를 통해, 모터의 회전속도를 변경할 수 있으며, VqTesting 변수를 통해, PWM의 Duty-ratio를 증가시킬 수 있습니다.

  SpeedRef 변수는 -0.95 ~ +0.95까지 변경해볼 수 있으며, VqTesting 변수는 0.0 ~ +1.0까지 변화시킬 수 있습니다.

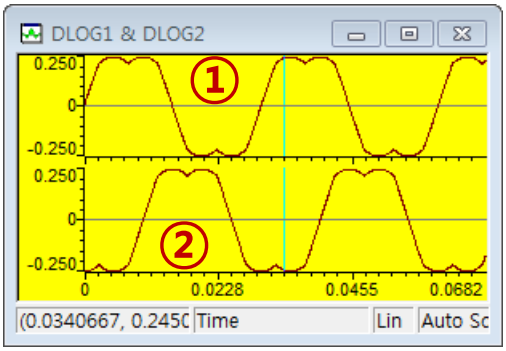
- **주요 관찰 대상 변수 → 없음**

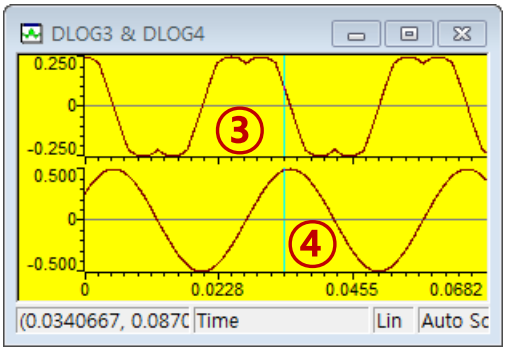
  이 단계에서는 모터를 직접 구동하지 않고, Feedback 되는 정보도 없기에, 별도로 관찰할 변수들은 없습니다. SMC150 개발보드의 테스트 홀을 통해, 프로그램이 구동되면서 출력되는 PWM 출력과 모터 드라이버 IC를 거쳐 모터에 전달될 U, V, W 상 출력을 계측장비로 확인해보실 수 있습니다.



SMC150 개발보드와 계측장비 간의 연결은 반드시! 모든 전원이 OFF 된 상태에서 이루어져야 하며, 전원이 인가된 상태에서 계측장비를 SMC150에 연결할 경우 계측장비의 측정포인터와 개발보드 사이에 Short로 인해 개발장비의 소손이 발생할 수 있으니 주의를 부탁 드립니다.

⑪ Graph Window를 통해서도 주요 변수들의 변화이력을 관찰할 수 있습니다. Build Level 1에서는 아래의 그림과 같은 화면을 확인하실 수 있으며, 1번부터 차례로 svgen_dq1.Ta, svgen_dq1.Tb, svgen_dq1.Tc, (svgen_dq1.Ta - svgen_dq1.Tb) 변수의 이력을 의미합니다.





⑫ 3장의 실습 끝내기 절차를 참조하여, 안전하게 예제 실행을 종료합니다.

2.4 실습 따라 하기 (Build Level 2)

① SMC150 개발보드에 DC+24V의 전원을 공급하고, JTAG 에뮬레이터를 통해, SMC150 개발보드에 장착된 프로세서 모듈과 PC의 Code Composer Studio를 연결합니다.

② SMC150 개발보드의 PSW1300 스위치를 "ON" 위치로 조작하여, 전원을 공급합니다. 단, DC-Link 전원 공급을 제어하는 PSW4000 스위치는 켜지 않습니다. (개발보드 조작과 관련한 세부내용은 제품 매뉴얼에서 자세히 확인하실 수 있습니다.)

③ Code Composer Studio에서 예제 프로젝트를 열고, PMSM_Sensored.h 파일에 정의된 Build Level이 "LEVEL2" 인지 확인합니다. (다르다면 LEVEL2로 수정합니다.)

④ 예제 프로젝트를 Build하고, 프로세서 모듈에 Output File을 Load 합니다.

⑤ Reset CPU → Restart → Enable Real-time Mode → Run(or Resume) 명령을 실행합니다.

⑥ 예제 프로젝트가 위치한 경로에 포함된 이미지 파일(*.png)을 참조하여, Watch Window에 관찰 및 조작할 변수를 등록하고, Graph Window를 설정합니다. CCS V3.3용 예제의 경우 GraphProperties1.png, GraphProperties2.png 파일을 참조하여 직접 Graph Window를 설정해야 하며, CCS V4.2.X / CCS V5.2.X용 예제는 Import 기능을 통해 미리 설정된 값들을 불러들일 수 있도록 GraphProperties1.graphPorp, GraphProperties2.graphPorp 파일이 포함되어 있습니다. Watch Window에 등록할 변수들은 CCS V3.3용 예제와 CCS V4.2.X용 예제 모두 이미지 파일로 제공되는 예시(WatchWindow_BuildLevel2.png)를 참조하여 직접 변수들을 등록해줘야 합니다.) 단, CCS V5.2.X의 경우는 미리 저장된 등록대상 변수들을 Watch Window(Expressions 탭)에 Import 할 수 있습니다. WatchWindow_BuildLevel2.txt 파일을 통해 관찰대상 변수들을 한번에 등록하십시오. – Expression 탭의 빈 공간을 마우스 우측버튼으로 클릭하면, 단축메뉴에서 Import 항목을 찾을 수 있습니다.

⑦ Watch Window와 Graph Window 모두 Continuous Refresh 옵션을 활성화 합니다.

⑧ DC-Link 전원 제어용 스위치(PSW4000)를 "ON" 위치로 조작합니다.

⑨ Watch Window 에 등록된 "EnableFlag" 변수를 0 에서 1 로 변경합니다. (EnableFlag 변수가 1 이 되면, 모터가 회전을 시작합니다.)

⑩ Build Level 2는 PMSM을 Open-loop로 구동하면서, ADC를 통해 U상과 V상의 전류가 올바르게 검출되는지 확인하는 단계 입니다. 이어지는 설명을 참조하여 Watch Window에 등록된 변수들을 관찰하고, 조작해봅니다.

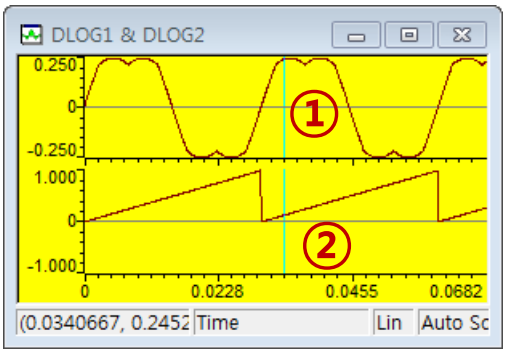
- **조작 대상 변수 → SpeedRef, VqTesting**

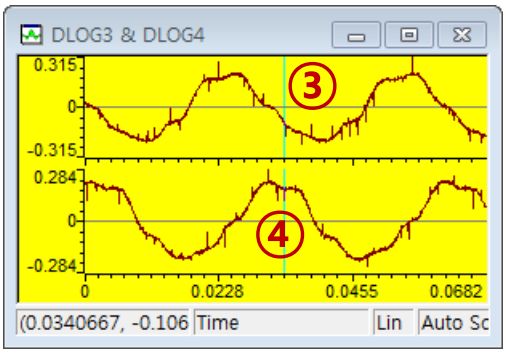
  SpeedRef 변수 값의 변화에 따라, RAMP_GEN 모듈이 모터의 회전자 위치에 해당되는 시뮬레이션 값을 만들어내고, 이를 기초로 모터가 Open-loop로 구동되게 됩니다. VqTesting 변수를 통해 모터에 전달되는 에너지를 조절할 수 있으며, 모터가 회전함에 따라 그래프 창을 통해 U상, V상의 전류가 올바르게 측정되는지 확인할 수 있습니다. (본 단계는 모터 회전자의 위치를 센서로 관찰하지 않고, 임의의 시뮬레이션 각도 값을 기초로 모터를 Open-loop 구동하는 단계 입니다. 따라서, VqTesting 변수에 의해 모터에 전달되는 에너지가 SpeedRef 변수와 Ramp Generation 모듈에 의해 시뮬레이션 되는 회전자의 회전속도를 유지하기에 부족할 경우, 모터가 회전을 정지하며 진동할 수 있습니다. 이 때는 SpeedRef 변수 값을 낮춰주거나, VqTesting 변수의 값을 조금 올려주고 모터의 회전자를 회전방향으로 살짝 돌려주게 되면, 모터의 회전자가 다시 회전자계를 따라 돌아가게 됩니다.)

- **주요 관찰 대상 변수 → ilg2_vdc1 구조체의 내부변수들**

  ilg2_vdc1은 TMS320F28X 프로세서의 ADC를 제어하는 S/W 모듈이며, U상과 V상의 상-전류와 DC-Link에 인가된 전압을 검출하도록 구성되어 있습니다. 각 채널에 대한 Gain 조절 변수(ImeasXGain)는 Q13 Format을 이용하며, Offset 조절 변수(ImeasXOffset)와 측정된 결과 데이터 저장용 변수(ImeasX)는 Q15 Format을 사용합니다. DC-Link 전압 측정용 변수들도 Gain 조절용 변수는 Q13 Format, Offset 조절 변수와 결과 저장용 변수는 Q15 Format을 사용합니다. (ilg2_vdc1이 검출한 U상, V상 전류 정보는 Clarke Transform S/W 모듈의 입력으로 사용됩니다.)

⑪ Graph Window를 통해서도 주요 변수들의 변화이력을 관찰할 수 있습니다. Build Level 2 에서는 아래의 그림과 같은 화면을 확인하실 수 있으며, 1번부터 차례로 svgen_dq1.Ta, rg1.Out, clarke1.As, clarke1.Bs 변수의 이력을 의미합니다.





⑫ 3장의 실습 끝내기 절차를 참조하여, 안전하게 예제 실행을 종료합니다.

2.5 실습 따라 하기 (Build Level 3)

① SMC150 개발보드에 DC+24V의 전원을 공급하고, JTAG 에뮬레이터를 통해, SMC150 개발보드에 장착된 프로세서 모듈과 PC의 Code Composer Studio를 연결합니다.

② SMC150 개발보드의 PSW1300 스위치를 "ON" 위치로 조작하여, 전원을 공급합니다. 단, DC-Link 전원 공급을 제어하는 PSW4000 스위치는 켜지 않습니다. (개발보드 조작과 관련한 세부내용은 제품 매뉴얼에서 자세히 확인하실 수 있습니다.)

③ Code Composer Studio에서 예제 프로젝트를 열고, PMSM_Sensored.h 파일에 정의된 Build Level이 "LEVEL3" 인지 확인합니다. (다르다면 LEVEL3으로 수정합니다.)

④ 예제 프로젝트를 Build하고, 프로세서 모듈에 Output File을 Load 합니다.

⑤ Reset CPU → Restart → Enable Real-time Mode → Run(or Resume) 명령을 실행합니다.

⑥ 예제 프로젝트가 위치한 경로에 포함된 이미지 파일(*.png)을 참조하여, Watch Window에 관찰 및 조작할 변수를 등록하고, Graph Window를 설정합니다. CCS V3.3용 예제의 경우 GraphProperties1.png, GraphProperties2.png 파일을 참조하여 직접 Graph Window를 설정해야 하며, CCS V4.2.X / CCS V5.2.X용 예제는 Import 기능을 통해 미리 설정된 값들을 불러들일 수 있도록 GraphProperties1.graphPorp, GraphProperties2.graphPorp 파일이 포함되어 있습니다. Watch Window에 등록할 변수들은 CCS V3.3용 예제와 CCS V4.2.X용 예제 모두 이미지 파일로 제공되는 예시(WatchWindow_BuildLevel3.png)를 참조하여 직접 변수들을 등록해줘야 합니다.) 단, CCS V5.2.X의 경우는 미리 저장된 등록대상 변수들을 Watch Window(Expressions 탭)에 Import 할 수 있습니다. WatchWindow_BuildLevel3.txt 파일을 통해 관찰대상 변수들을 한번에 등록하십시오. – Expression 탭의 빈 공간을 마우스 우측버튼으로 클릭하면, 단축메뉴에서 Import 항목을 찾을 수 있습니다.

⑦ Watch Window와 Graph Window 모두 Continuous Refresh 옵션을 활성화 합니다.

⑧ DC-Link 전원 제어용 스위치(PSW4000)를 "ON" 위치로 조작합니다.

⑨ Watch Window 에 등록된 "EnableFlag" 변수를 0 에서 1 로 변경합니다. (EnableFlag 변수가 1 이 되면, 모터가 회전을 시작합니다.)

⑩ Build Level 3은 PMSM을 Open-loop로 구동하면서, ADC를 통해 검출된 U, V상 전류정보에 PI 제어모듈을 연결하고, 전류제어가 올바르게 되는지 확인하는 단계 입니다. 이어지는 설명을 참조하여 Watch Window에 등록된 변수들을 관찰하고, 조작해봅니다.

- **조작 대상 변수 → SpeedRef, IqRef**

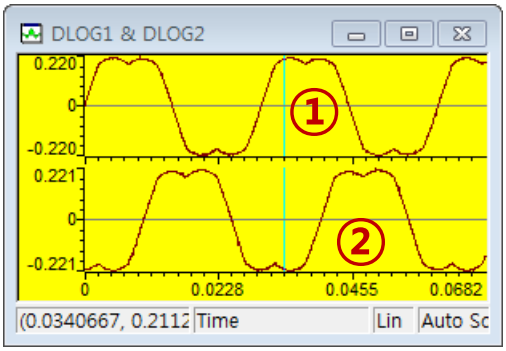
  SpeedRef 변수 값의 변화에 따라, RAMP_GEN 모듈이 모터의 회전자 위치에 해당되는 시뮬레이션 값을 만들어내고, 이를 기초로 모터가 Open-loop로 구동되게 됩니다. IqRef 변수를 통해 모터에 전달되는 에너지를 조절할 수 있으며, IqRef와 Q축 전류제어용 PI 제어모듈에 의해 지령 전류를 추종하도록 제어가 이루어집니다. 모터가 회전함에 따라 그래프 창을 통해 U상 전류가 올바르게 제어되는지 확인할 수 있습니다. IqRef 값은 0.0 ~ +0.5까지의 범위에서 조작해주십시오. 해당 범위 이상의 값으로 조작될 경우, SMC150 개발보드의 과-전류 Trip 회로가 동작하여, 모터 드라이버 IC에 전달되는 PWM 출력이 차단될 수 있습니다. (본 단계는 모터 회전자의 위치를 센서로 관찰하지 않고, 임의의 시뮬레이션 각도 값을 기초로 모터를 Open-loop 구동하는 단계 입니다. 따라서, 회전축을 손으로 잡거나, 부하를 연결하여, IqRef 변수와 PI 제어 모듈에 의해 모터에 전달되는 에너지가 SpeedRef 변수와 Ramp Generation 모듈에 의해 시뮬레이션 되는 회전자의 회전속도를 유지하기에 부족할 경우, 모터가 회전을 정지하며 진동할 수 있습니다. 회전자에 걸린 부하를 제거하거나, 잡고 있던 손을 때면, 모터의 회전자가 다시 회전자계를 따라 돌아가게 됩니다.)

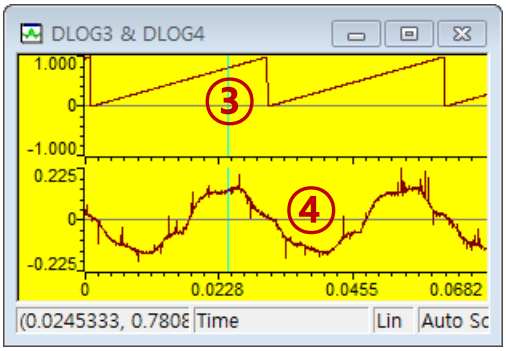
- **주요 관찰 대상 변수 → ilg2_vdc1 구조체와 pid1_iq 구조체의 내부변수들**

  ilg2_vdc1은 TMS320F28X 프로세서의 ADC를 제어하는 S/W 모듈이며, U상과 V상의 상-전류와 DC-Link에 인가된 전압을 검출하도록 구성되어 있습니다. 각 채널에 대한 Gain 조절 변수(ImeasXGain)는 Q13 Format을 이용하며, Offset 조절 변수(ImeasXOffset)와 측정된 결과 데이터 저장용 변수(ImeasX)는 Q15 Format을 사용합니다. DC-Link 전압 측정용 변수들도 Gain 조절용 변수는 Q13 Format, Offset 조절 변수와 결과 저장용 변수는 Q15 Format을 사용합니다. (ilg2_vdc1이 검출한 U상, V상 전류 정보는 Clarke Transform S/W 모듈의 입력으로 사용됩니다.)

pid1_iq는 IqRef 변수 지령과 ilg2_vdc1이 검출한 U, V상 전류 정보로 산출된 정보를 Clarke 토대로 모터에 전달될 Q축 전류지령을 제어하는 PI 제어용 S/W 모듈 입니다. IQmath 헤더파일에서 정의된 Global Q Format을 기초로 동작하며, Proportional Gain(Kp), Integral Gain(Ki), Derivative Gain(Kd) 외에 PI 제어기의 적분 값이 필요 이상으로 누적됨을 막아주는(Anti-windup) Integral Correction Gain(Kc)을 설정할 수 있습니다. (예제에서 Derivative Gain을 사용하지 않기에, Derivative Output에 대한 연산 코드는 주석 처리되어 있습니다.)

⑪ Graph Window를 통해서도 주요 변수들의 변화이력을 관찰할 수 있습니다. Build Level 3에서는 아래의 그림과 같은 화면을 확인하실 수 있으며, 1번부터 차례로 svgen_dq1.Ta, svgen_dq1.Tb, rg1.Out, clarke1.As 변수의 이력을 의미합니다.





⑫ 3장의 실습 끝내기 절차를 참조하여, 안전하게 예제 실행을 종료합니다.

2.6 실습 따라 하기 (Build Level 4)

① SMC150 개발보드에 DC+24V의 전원을 공급하고, JTAG 에뮬레이터를 통해, SMC150 개발보드에 장착된 프로세서 모듈과 PC의 Code Composer Studio를 연결합니다.

② SMC150 개발보드의 PSW1300 스위치를 "ON" 위치로 조작하여, 전원을 공급합니다. 단, DC-Link 전원 공급을 제어하는 PSW4000 스위치는 켜지 않습니다. (개발보드 조작과 관련한 세부내용은 제품 매뉴얼에서 자세히 확인하실 수 있습니다.)

③ Code Composer Studio에서 예제 프로젝트를 열고, PMSM_Sensored.h 파일에 정의된 Build Level이 "LEVEL4" 인지 확인합니다. (다르다면 LEVEL4로 수정합니다.)

④ 예제 프로젝트를 Build하고, 프로세서 모듈에 Output File을 Load 합니다.

⑤ Reset CPU → Restart → Enable Real-time Mode → Run(or Resume) 명령을 실행합니다.

⑥ 예제 프로젝트가 위치한 경로에 포함된 이미지 파일(*.png)을 참조하여, Watch Window에 관찰 및 조작할 변수를 등록하고, Graph Window를 설정합니다. CCS V3.3용 예제의 경우 GraphProperties1.png, GraphProperties2.png 파일을 참조하여 직접 Graph Window를 설정해야 하며, CCS V4.2.X / CCS V5.2.X용 예제는 Import 기능을 통해 미리 설정된 값들을 불러들일 수 있도록 GraphProperties1.graphPorp, GraphProperties2.graphPorp 파일이 포함되어 있습니다. Watch Window에 등록할 변수들은 CCS V3.3용 예제와 CCS V4.2.X용 예제 모두 이미지 파일로 제공되는 예시(WatchWindow_BuildLevel4.png)를 참조하여 직접 변수들을 등록해줘야 합니다.) 단, CCS V5.2.X의 경우는 미리 저장된 등록대상 변수들을 Watch Window(Expressions 탭)에 Import 할 수 있습니다. WatchWindow_BuildLevel4.txt 파일을 통해 관찰대상 변수들을 한번에 등록하십시오. – Expression 탭의 빈 공간을 마우스 우측버튼으로 클릭하면, 단축메뉴에서 Import 항목을 찾을 수 있습니다.

⑦ Watch Window와 Graph Window 모두 Continuous Refresh 옵션을 활성화 합니다.

⑧ DC-Link 전원 제어용 스위치(PSW4000)를 "ON" 위치로 조작합니다.

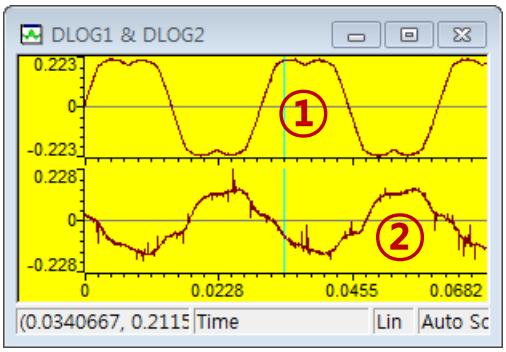
⑨ Watch Window 에 등록된 "EnableFlag" 변수를 0 에서 1 로 변경합니다. (EnableFlag 변수가 1 이 되면, 모터가 회전을 시작합니다.)

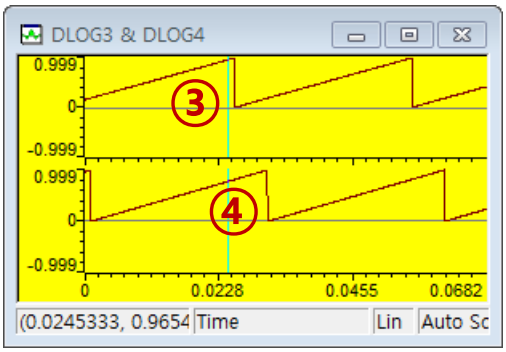
⑩ Build Level 4는 PMSM을 Open-loop로 구동하면서, TMS320F28X 프로세서의 EQEP 모듈로 엔코더로부터의 신호를 해석하고, 올바르게 위치정보를 검출해내는지 확인하는 단계입니다. 이어지는 설명을 참조하여 Watch Window에 등록된 변수들을 관찰하고, 조작해 봅니다.

- **조작 대상 변수 → SpeedRef, IqRef**
  SpeedRef, IqRef 변수의 조작과 그에 대한 설명은 Build Level 3과 동일합니다.

- **주요 관찰 대상 변수 → qep1 구조체의 내부변수들 (Watch Window에 등록)**
  qep1을 통해, TMS320F28X 프로세서의 EQEP 모듈이 모터와 연결된 Incremental Encoder로부터 전달되는 펄스신호를 해석하여 산출해낸, 위치정보를 확인하실 수 있습니다. ElecTheta는 전기각을 의미하며, MechTheta는 실제 회전축의 기계각을 의미합니다. 본 예제에서 사용된 PMSM은 8poles(4pole pair) 이므로, 기계각 1회전 당 전기각이 4회 회전하게 됩니다. ElecTheta 변수와 MechTheta 변수는 모두 IQmath 헤더파일에서 정의한 Global Q Format을 이용합니다. (EQEP 모듈을 통해 Encoder 신호를 해석해내지만, 모터 구동은 여전히 Build Level 3단계와 마찬가지로 Open-loop로 이루어집니다.)

⑪ Graph Window를 통해서도 주요 변수들의 변화이력을 관찰할 수 있습니다. Build Level 4에서는 아래의 그림과 같은 화면을 확인하실 수 있으며, 1번부터 차례로 svgen_dq1.Ta, clarke1.As, qep1.ElecTheta, rg1.Out 변수의 이력을 의미합니다.





⑫ 3장의 실습 끝내기 절차를 참조하여, 안전하게 예제 실행을 종료합니다.

2.7 실습 따라 하기 (Build Level 5)

① SMC150 개발보드에 DC+24V의 전원을 공급하고, JTAG 에뮬레이터를 통해, SMC150 개발보드에 장착된 프로세서 모듈과 PC의 Code Composer Studio를 연결합니다.

② SMC150 개발보드의 PSW1300 스위치를 "ON" 위치로 조작하여, 전원을 공급합니다. 단, DC-Link 전원 공급을 제어하는 PSW4000 스위치는 켜지 않습니다. (개발보드 조작과 관련한 세부내용은 제품 매뉴얼에서 자세히 확인하실 수 있습니다.)

③ Code Composer Studio에서 예제 프로젝트를 열고, PMSM_Sensored.h 파일에 정의된 Build Level이 "LEVEL5" 인지 확인합니다. (다르다면 LEVEL5로 수정합니다.)

④ 예제 프로젝트를 Build하고, 프로세서 모듈에 Output File을 Load 합니다.

⑤ Reset CPU → Restart → Enable Real-time Mode → Run(or Resume) 명령을 실행합니다.

⑥ 예제 프로젝트가 위치한 경로에 포함된 이미지 파일(*.png)을 참조하여, Watch Window에 관찰 및 조작할 변수를 등록하고, Graph Window를 설정합니다. CCS V3.3용 예제의 경우 GraphProperties1.png, GraphProperties2.png 파일을 참조하여 직접 Graph Window를 설정해야 하며, CCS V4.2.X / CCS V5.2.X용 예제는 Import 기능을 통해 미리 설정된 값들을 불러들일 수 있도록 GraphProperties1.graphPorp, GraphProperties2.graphPorp 파일이 포함되어 있습니다. Watch Window에 등록할 변수들은 CCS V3.3용 예제와 CCS V4.2.X용 예제 모두 이미지 파일로 제공되는 예시(WatchWindow_BuildLevel5.png)를 참조하여 직접 변수들을 등록해줘야 합니다.) 단, CCS V5.2.X의 경우는 미리 저장된 등록대상 변수들을 Watch Window(Expressions 탭)에 Import 할 수 있습니다. WatchWindow_BuildLevel5.txt 파일을 통해 관찰대상 변수들을 한번에 등록하십시오. – Expression 탭의 빈 공간을 마우스 우측버튼으로 클릭하면, 단축메뉴에서 Import 항목을 찾을 수 있습니다.

⑦ Watch Window와 Graph Window 모두 Continuous Refresh 옵션을 활성화 합니다.

⑧ DC-Link 전원 제어용 스위치(PSW4000)를 "ON" 위치로 조작합니다.

⑨ Watch Window 에 등록된 "EnableFlag" 변수를 0 에서 1 로 변경합니다. (EnableFlag 변수가 1 이 되면, 모터가 회전을 시작합니다.)

⑩ Build Level 5는 1~4까지의 Build Level에서 확인한, 전류제어 Loop, 엔코더의 위치정보 등을 기초로 하여, PMSM을 Closed-loop로 제어하는 단계이며, Q축 전류제어 Loop 외부에 속도제어 PI 제어 Loop를 추가하여, 사용자의 속도지령을 추종하도록 합니다. 이어지는 설명을 참조하여 Watch Window에 등록된 변수들을 관찰하고, 조작해봅니다.

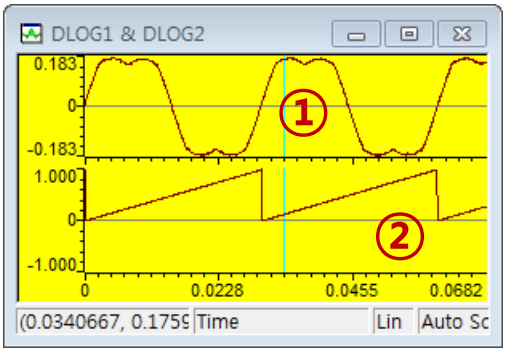
- **조작 대상 변수 → SpeedRef**

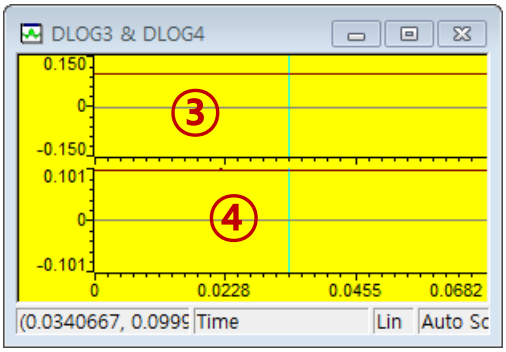
  SpeedRef 변수를 -0.95 ~ +0.95의 범위에서 조작하여, 모터의 회전속도를 변화시켜볼 수 있습니다. SpeedRef 변수는 속도제어용 PI 제어기의 지령입력(Reference)으로 사용되며, 이 속도제어용 PI 제어기의 출력은 다시, Q축 전류제어용 PI 제어기의 지령입력으로 이용됩니다. 결국, 사용자의 SpeedRef 변수 조작에 따라, Q축에 전달되는 전류가 제어되며, 모터의 회전속도가 사용자의 속도지령에 추종하도록 제어하게 됩니다.

- **주요 관찰 대상 변수 → speed1 구조체와 pid1_spd 구조체의 내부변수들**

  speed1을 통해, 현재 모터의 회전속도 정보를 확인할 수 있습니다. qep1이 Encoder의 펄스로부터 해석한 위치정보를 토대로 회전속도를 산출하며, 사용자가 정의한 모터의 Base RPM에 대한 현 속도의 비율(speed1.Speed / IQmath 헤더의 Global Q Format 사용)과 이를 RPM 단위로 환산한 결과(speed1.SpeedRPM)를 보여줍니다.

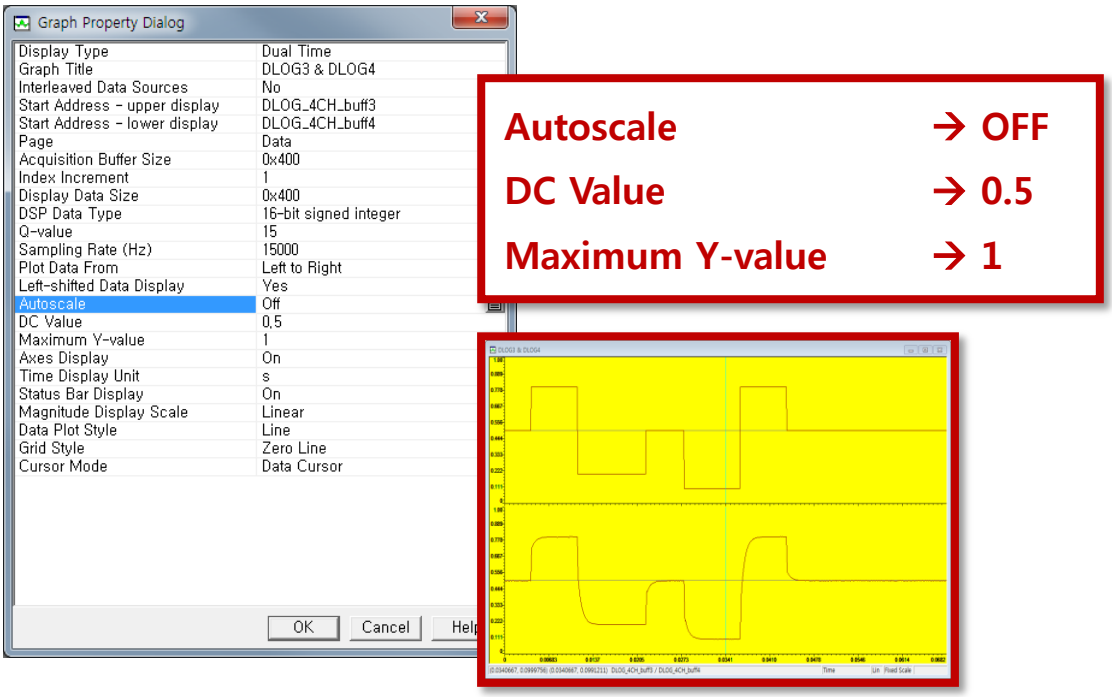
⑪ Graph Window를 통해서도 주요 변수들의 변화이력을 관찰할 수 있습니다. Build Level 5 에서는 아래의 그림과 같은 화면을 확인하실 수 있으며, 1번부터 차례로 svgen_dq1.Ta, speed1.ElecTheta, pid1_spd.Ref, pid1_spd.Fdb 변수의 이력을 의미합니다.





⑫ SpeedKp, SpeedKi, dlog.prescalar 변수를 Watch Window에 등록합니다.

- SpeedKp 변수는 속력제어용 PI 제어기의 Proportional Gain을 의미하며, SpeedKi는 Integral Gain을 나타냅니다. 이 값들을 다르게 설정한 후, SpeedRef 변수를 변경해보면, 달라진 Gain에 따라 지령속도를 추종하는 형태가 달라짐을 확인할 수 있습니다. (Watch Window에 등록된 dlog.prescalar 변수의 값을 "500"으로 변경하고, "DLOG3 & DLOG4" Graph Window를 Auto Scale이 아닌 Fixed Scale로 설정하시면, 추종과정의 속력변화 History를 보다 확실히 확인하실 수 있습니다.)



⑬ 3장의 실습 끝내기 절차를 참조하여, 안전하게 예제 실행을 종료합니다.

2.8 실습 따라 하기 (Build Level 6)

① SMC150 개발보드에 DC+24V의 전원을 공급하고, JTAG 에뮬레이터를 통해, SMC150 개발보드에 장착된 프로세서 모듈과 PC의 Code Composer Studio를 연결합니다.

② SMC150 개발보드의 PSW1300 스위치를 "ON" 위치로 조작하여, 전원을 공급합니다. 단, DC-Link 전원 공급을 제어하는 PSW4000 스위치는 켜지 않습니다. (개발보드 조작과 관련한 세부내용은 제품 매뉴얼에서 자세히 확인하실 수 있습니다.)

③ Code Composer Studio에서 예제 프로젝트를 열고, PMSM_Sensored.h 파일에 정의된 Build Level이 "LEVEL6" 인지 확인합니다. (다르다면 LEVEL6으로 수정합니다.)

④ 예제 프로젝트를 Build하고, 프로세서 모듈에 Output File을 Load 합니다.

⑤ Reset CPU → Restart → Enable Real-time Mode → Run(or Resume) 명령을 실행합니다.

⑥ 예제 프로젝트가 위치한 경로에 포함된 이미지 파일(*.png)을 참조하여, Watch Window에 관찰 및 조작할 변수를 등록하고, Graph Window를 설정합니다. CCS V3.3용 예제의 경우 GraphProperties1.png, GraphProperties2.png 파일을 참조하여 직접 Graph Window를 설정해야 하며, CCS V4.2.X / CCS V5.2.X용 예제는 Import 기능을 통해 미리 설정된 값들을 불러들일 수 있도록 GraphProperties1.graphPorp, GraphProperties2.graphPorp 파일이 포함되어 있습니다. Watch Window에 등록할 변수들은 CCS V3.3용 예제와 CCS V4.2.X용 예제 모두 이미지 파일로 제공되는 예시(WatchWindow_BuildLevel6.png)를 참조하여 직접 변수들을 등록해줘야 합니다.) 단, CCS V5.2.X의 경우는 미리 저장된 등록대상 변수들을 Watch Window(Expressions 탭)에 Import 할 수 있습니다. WatchWindow_BuildLevel6.txt 파일을 통해 관찰대상 변수들을 한번에 등록하십시오. – Expression 탭의 빈 공간을 마우스 우측버튼으로 클릭하면, 단축메뉴에서 Import 항목을 찾을 수 있습니다.

⑦ Watch Window와 Graph Window 모두 Continuous Refresh 옵션을 활성화 합니다.

⑧ DC-Link 전원 제어용 스위치(PSW4000)를 "ON" 위치로 조작합니다.

⑨ Watch Window 에 등록된 "EnableFlag" 변수를 0 에서 1 로 변경합니다. (EnableFlag 변수가 1 이 되면, 모터가 회전을 시작합니다.)

⑩ Build Level 6은 Build Level 5와 유사한 Closed-loop 단계이며, Q축 전류제어 Loop 외부에 속도 PI 제어 Loop 대신, 위치제어 Loop가 구성되어 있어, 모터의 회전자가 사용자의 위치지령을 추종하도록 합니다. 이어지는 설명을 참조하여 Watch Window에 등록된 변수들을 관찰하고, 조작해봅니다.

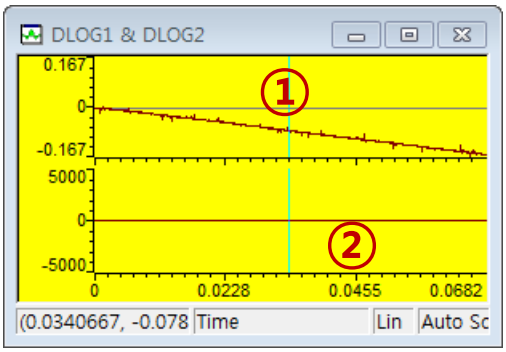
- **조작 대상 변수 → PositionRef**

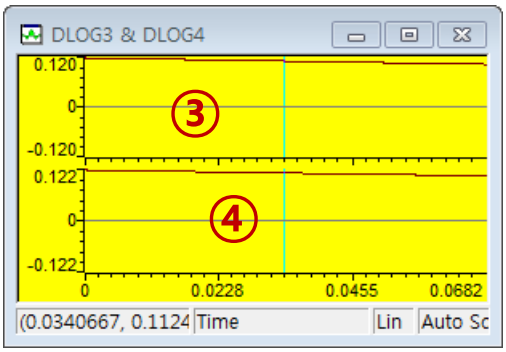
  PositionRef 변수를 0.05 ~ +0.95의 범위에서 조작하여, 모터 회전축의 위치를 제어해볼 수 있습니다. PositionRef 변수는 위치제어용 PI 제어기의 지령입력(Reference)으로 사용되며, 이 위치제어용 PI 제어기의 출력은 다시, Q축 전류제어용 PI 제어기의 지령입력으로 이용됩니다. 결국, 사용자의 PositionRef 변수 조작에 따라, Q축에 전달되는 전류가 제어되며, 모터 회전축의 위치가 사용자의 위치지령에 추종하도록 제어하게 됩니다.

- **주요 관찰 대상 변수 → qep1 구조체와 pid1_pos 구조체의 내부변수들**

  Build Level 6은 이전의 Level 5와 실습의 진행 및 모터의 회전특성 관찰 방법이 유사하며, qep1.MechTheta 변수 값을 통해, 현재 모터 회전축 위치를 확인할 수 있고, pid1_pos 구조체의 내부변수들을 통해 위치 PI 제어 Loop의 동작을 확인할 수 있습니다.

⑪ Graph Window를 통해서도 주요 변수들의 변화이력을 관찰할 수 있습니다. Build Level 6에서는 아래의 그림과 같은 화면을 확인하실 수 있으며, 1번부터 차례로 svgen_dq1.Ta, speed1.ElecTheta, pid1_pos.Ref, pid1_pos.Fdb 변수의 이력을 의미합니다.





⑫ 3장의 실습 끝내기 절차를 참조하여, 안전하게 예제 실행을 종료합니다.

3 실습 끝내기 절차

① **Watch Window에 등록된 "EnableFlag" 변수를 0으로 변경합니다.** (변경 후 모터가 정지하는 것을 확인합니다.)

② **SMC150 개발보드의 DC-Link 전원 제어용 스위치(PSW4000)를 "OFF" 위치로 변경하여, 모터 드라이버 IC에 공급되는 전원을 차단합니다.**

③ Code Composer Studio에서 Halt(or Suspend) 명령으로 프로그램 실행을 멈추고, Real-time Mode도 해제합니다.

④ Code Composer Studio를 종료하고, SMC150 개발보드에 공급된 전원을 차단합니다.

⑤ JTAG 에뮬레이터와 프로세서 모듈을 분리합니다.